

Architecture Documentation: MockInterview.ai

AI-powered adaptive technical interview preparation platform

1 Problem Summary and User Journey

1.1 Problem Statement

Candidates preparing for technical interviews often have limited access to affordable, personalised, and on-demand practice. Human coaching is expensive, while generic preparation tools do not adapt to a candidate's resume, skill gaps, target role, or real-time performance.

A major gap is that most systems evaluate only *what* a candidate answers, but not *how* the candidate communicates. MockInterview.ai addresses this gap by combining resume intelligence, adaptive interview generation, technical evaluation, and multimodal feedback.

1.2 Proposed Solution

MockInterview.ai is an end-to-end AI interview preparation platform that ingests a candidate's resume, extracts skills and weak areas, recommends suitable roles and jobs, conducts an adaptive five-question interview, evaluates answers using technical, audio, and video signals, and generates a personalised feedback report.

1.3 User Journey

1. **Upload Resume** – Candidate uploads a PDF or plain-text resume.
2. **Resume Intelligence** – System extracts skills, experience, projects, and weak areas.
3. **Role Recommendations** – Candidate is compared with engineering role profiles.
4. **Job Recommendations** – Candidate is matched against curated job listings.
5. **Adaptive Interview** – Candidate answers five questions with dynamic difficulty adjustment.
6. **Multimodal Scoring** – Answers are scored using technical, audio, and video signals.
7. **Feedback Report** – Final report includes overall score, radar chart, breakdown, and learning plan.

2 High-Level Architecture

The system follows a layered architecture with a React frontend, FastAPI backend, agent layer, service layer, optional LLM layer, and persistence layer.

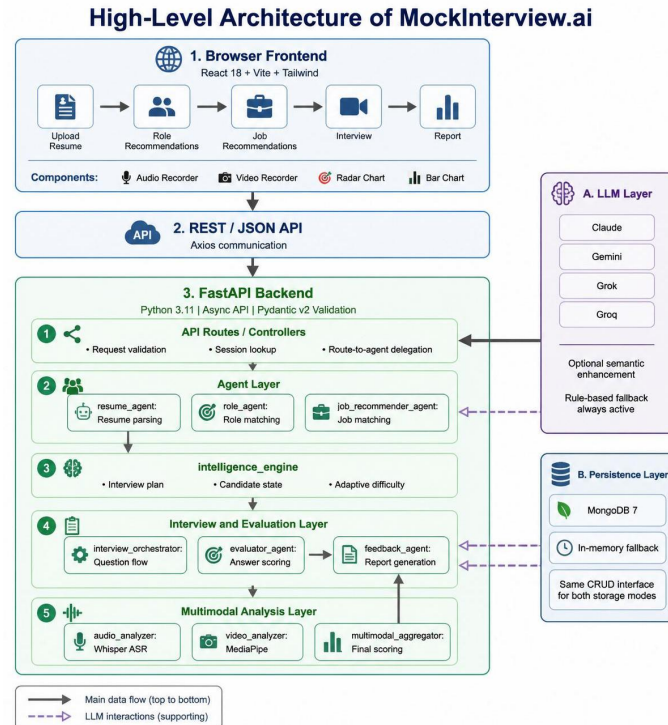


Figure 1: High-level architecture of MockInterview.ai

2.1 Architecture Overview

The architecture is organised into three major areas: **frontend layer**, **backend layer**, and **supporting layers**. The frontend handles resume upload, interview flow, and report visualisation. The backend handles request validation, session management, agent orchestration, scoring, and feedback generation. Supporting layers provide optional LLM enhancement and persistence through MongoDB or in-memory fallback.

2.2 Frontend and API

The frontend is a React 18 single-page application built with Vite and Tailwind CSS. It provides the resume upload page, role recommendation page, job recommendation page, interview page, and feedback report page. It also includes browser-based components such as Audio Recorder, Video Recorder, Radar Chart, and Bar Chart. The frontend communicates with the backend using RESTful JSON APIs through Axios. The API layer sends resume files, candidate data, interview answers, audio/video inputs, and retrieves recommendations and final reports.

2.3 Backend Components

The backend is implemented using FastAPI with Python 3.11. It is designed as an asynchronous API service, with Pydantic v2 used for request and response validation.

API Routes and Controllers: Routes receive requests, validate inputs, perform session lookup, and delegate work to the correct agent or service.

Agent Layer: The main agents are `resume_agent`, `role_agent`, `job_recommender_agent`, `evaluator_agent`, and `feedback_agent`. Agents are stateless and return structured outputs.

Intelligence Engine: The `intelligence_engine` manages interview plan generation, candidate state, skill gap awareness, adaptive difficulty, and role-aware question selection.

Interview and Evaluation Layer: The `interview_orchestrator` controls the interview flow, while `evaluator_agent` scores candidate answers and `feedback_agent` generates the final report.

Multimodal Analysis Layer: The `audio_analyzer` uses Whisper ASR, the `video_analyzer` uses MediaPipe, and the `multimodal_aggregator` combines technical, communication, confidence, engagement, and role-fit scores.

2.4 LLM and Persistence Layers

The LLM layer is optional. When `USE_LLM=true` and a valid API key is available, the platform can use Claude, Gemini, Grok, or Groq-hosted models for semantic enhancement in resume understanding, answer evaluation, and feedback generation. Rule-based fallbacks remain active at all times.

MongoDB 7 is the primary persistence layer. If MongoDB is unavailable, the backend falls back to an in-memory store. Both storage modes expose the same CRUD interface, allowing the rest of the application to remain unchanged.

3 Key Design Choices and Trade-offs

Choice	Rationale	Trade-off
Rule-based fallback for every agent	Platform works without API keys and is reliable for demos/local development.	Less nuanced than LLM outputs; static question banks need maintenance.
Multi-provider LLM abstraction	Claude, Gemini, Groq, or other providers can be swapped without changing agent code.	Requires response normalisation across providers.
MongoDB with in-memory fallback	Enables quick local development even without a database.	In-memory state is lost after backend restart.
Threshold-based adaptive difficulty	Simple, deterministic, and easy to debug.	Less precise than Item Response Theory or Bayesian Knowledge Tracing.
MediaPipe-based video metrics	CPU-friendly and avoids cloud vision dependency.	Face presence and movement are proxy signals, not true behavioural classifiers.
Whisper-based audio analysis	Strong open-source transcription capability.	First-run model download can add latency.
LLM timeout protection	Prevents interview sessions from hanging.	Timeout falls back to lower-quality rule-based output.
Docker Compose deployment	Enables one-command local deployment.	Initial build can be slow and is not full autoscaling production setup.

Table 1: Major design choices and trade-offs

4 Scoring Approach and Aggregation

4.1 Per-Answer Technical Scoring

Each answer is evaluated independently by the `evaluator_agent` using three axes: correctness, depth, and technical accuracy.

Axis	Method
Correctness	Measures how many expected points are covered by the answer.
Depth	Uses answer length, examples, explanation quality, and elaboration signals.
Technical Accuracy	Combines correctness and depth to estimate technical soundness.

Table 2: Per-answer evaluation axes

Answer Score = (Correctness × 0.50) + (Depth × 0.30) + (Technical Accuracy × 0.20)

When LLM mode is enabled, semantic scoring is used to identify covered and missing points more accurately.

4.2 Multimodal Final Score

$$\text{Final Score} = (\text{Technical} \times 0.45) + (\text{Communication} \times 0.20) + (\text{Confidence} \times 0.15) + (\text{Engagement} \times 0.10) + (\text{Role Fit} \times 0.10)$$

Dimension	Primary Source	Fallback
Technical	Average of all answer scores	Same
Communication	Audio clarity score from Whisper	Answer-length proxy
Confidence	Audio confidence adjusted for hesitation/pauses	Technical-depth average
Engagement	MediaPipe face-detection rate	Session-duration proxy
Role Fit	Candidate experience vs role seniority	Same

Table 3: Multimodal scoring dimensions

4.3 Role Scoring

The `role_agent` compares the candidate profile against role profiles using core skill overlap, bonus/adjacent skill coverage, project evidence, experience-level affinity, and skill breadth bonus. The top matching roles are returned with scores, matched skills, and evidence bullets.

5 Limitations, Assumptions, and Next Steps

5.1 Limitations

- **Video proxies, not classifiers** – Eye contact and stress are approximated using face centring and movement.
- **Whisper latency** – First audio submission may trigger model download and delay.
- **Ephemeral sessions without MongoDB** – In-memory data is lost after restart.
- **English-only pipeline** – Question banks, rubrics, and prompts currently assume English.
- **Five-question sessions** – Deeper assessment requires longer or repeated sessions.
- **Static job listings** – Live job APIs are not active unless credentials are configured.
- **No authentication** – Candidate sessions rely on locally stored candidate IDs.

5.2 Assumptions

- Resumes are in English and follow conventional structures.
- Candidate devices have a working microphone and webcam for optional multimodal features.
- LLM API keys and Whisper models are optional.
- The platform should degrade gracefully when LLMs, audio, video, or MongoDB are unavailable.

5.3 Next Steps

- Replace threshold-based adaptation with Bayesian Knowledge Tracing or Item Response Theory.
- Add dedicated gaze estimation using MediaPipe Iris, dlib, or a pretrained vision model.
- Implement JWT-based user accounts and longitudinal interview history.
- Expand the question bank with community and LLM-generated questions.
- Add real-time streaming feedback using Server-Sent Events.
- Integrate live job APIs such as Adzuna or LinkedIn.
- Export feedback reports as PDFs.
- Add rate limiting, quota management, and production scaling.